

Guia de Deploy — ES-Builder

Boas práticas para configurar seu projeto para deploy automático

Como funciona o deploy

Cada grupo tem seu projeto servido em um subpath exclusivo:

```
http://<servidor>/portal/      → grupo portal
http://<servidor>/projeto-a/   → grupo A
http://<servidor>/projeto-b/   → grupo B
...
```

O deploy é automático: ao fazer push na branch `deploy`, o sistema detecta a mudança, reconstroi as imagens Docker e reinicia os containers.

Estrutura esperada do repositório

```
seu-repo/
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
├── frontend/
│   ├── Dockerfile
│   ├── nginx.conf
│   ├── package.json
│   └── src/
```

1. Frontend — Dockerfile

O Dockerfile do frontend precisa declarar e usar o `VITE_BASE_PATH` como build argument:

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .

# ✅ Obrigatório: declara e expõe o base path para o Vite
ARG VITE_BASE_PATH=/
ENV VITE_BASE_PATH=$VITE_BASE_PATH

RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

2. Frontend — vite.config.ts

Configure o `base` para usar a variável de ambiente:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  base: process.env.VITE_BASE_PATH || '/',
})
```

3. Frontend — src/vite-env.d.ts

Garanta que este arquivo existe na pasta `src/` para que o TypeScript reconheça

```
import.meta.env:
```

```
/// <reference types="vite/client" />
```

4. Frontend — React Router

Configure o `basename` do router com o base path:

```
// App.tsx
<BrowserRouter basename={import.meta as any}.env?.VITE_BASE_PATH ||
  '/'>
  <Routes>
    ...
  </Routes>
</BrowserRouter>
```

5. Frontend — Axios (chamadas de API)

Configure o `baseUrl` do axios com o prefixo do subpath. Geralmente em `src/services/api.ts` ou similar:

```
const BASE = (import.meta as any).env?.VITE_BASE_PATH?.replace(/\/$/,
  '') || '';

const api = axios.create({
  baseUrl: `${BASE}/api`,
  headers: { 'Content-Type': 'application/json' },
});
```

Se houver redirecionamento manual via `window.location.href`, inclua o prefixo:

```
// ❌ errado
window.location.href = '/login';

// ✅ correto
window.location.href = `${BASE}/login`;
```

6. Frontend — Assets estáticos (imagens, SVGs)

Caminhos absolutos em strings JSX **não** recebem o base path automaticamente.

```
// ❌ errado – vai para /cbiot-ufrgs.svg (404)


// ✅ opção 1 – importar como módulo (Vite resolve automaticamente)
import logo from '/cbiot-ufrgs.svg';
<img src={logo} />

// ✅ opção 2 – usar a variável de ambiente
<img src={`${(import.meta as any).env?.VITE_BASE_PATH || '/'}cbiot-
ufrgs.svg`} />
```

7. Frontend — nginx.conf

O container do frontend precisa de um `nginx.conf` com `try_files` para funcionar como SPA:

```
server {  
    listen 80;  
    root /usr/share/nginx/html;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
}
```

8. Backend — Dockerfile

Exemplo de Dockerfile funcional para NestJS/Express com Prisma:

```
FROM node:20-alpine  
WORKDIR /app  
  
RUN apk add --no-cache openssl  
  
COPY package*.json ./  
RUN npm install --include=dev  
  
COPY . .  
RUN npx prisma generate  
  
EXPOSE 3000  
CMD ["npm", "run", "start:dev"]
```

Atenção: o `npx prisma generate` requer que `prisma/schema.prisma` esteja disponível no contexto de build. O contexto deve ser a pasta `backend/`, não a raiz do repositório.

9. Branch `deploy`

O watcher monitora exclusivamente a branch `deploy`. Seu fluxo de trabalho deve ser:

```
# desenvolvimento normal na branch main
git add .
git commit -m "feat: nova funcionalidade"
git push origin main

# quando quiser fazer deploy
git checkout deploy
git merge main
git push origin deploy
git checkout main
```

Ou via PR do GitHub: abra um Pull Request de `main` → `deploy`.

10. Checklist antes de fazer push na branch deploy

- ☐ `frontend/Dockerfile` tem `ARG VITE_BASE_PATH` e `ENV VITE_BASE_PATH`
- ☐ `vite.config.ts` usa `process.env.VITE_BASE_PATH`
- ☐ `src/vite-env.d.ts` existe com `/// <reference types="vite/client" />`
- ☐ `<BrowserRouter>` tem `basename` configurado
- ☐ `axios.create({ baseURL })` usa `VITE_BASE_PATH` como prefixo
- ☐ Assets estáticos não usam paths absolutos hardcodados
- ☐ `frontend/nginx.conf` existe com `try_files`
- ☐ Backend tem `prisma/schema.prisma` acessível no contexto de build

Erros comuns e soluções

Sintoma	Causa	Solução
Tela branca, assets com 404	<code>base</code> não configurado no Vite	Configurar <code>vite.config.ts</code> e <code>Dockerfile</code>

Tela branca, assets com 200 e MIME error	<code>VITE_BASE_PATH</code> não declarado no Dockerfile	Adicionar <code>ARG</code> e <code>ENV</code> antes do <code>RUN npm run build</code>
Cor de fundo mas sem conteúdo	<code>BrowserRouter</code> sem <code>basename</code>	Adicionar <code>basename</code> no Router
Erro ao conectar ao backend	<code>axios.baseURL</code> sem prefixo do subpath	Configurar <code>baseURL</code> com <code>VITE_BASE_PATH</code>
Build falha com "schema.prisma not found"	Contexto de build incorreto	Verificar <code>context</code> em <code>config/projeto-x.json</code>
Logo/imagem com 404	Path absoluto hardcodado em JSX	Importar como módulo ou usar variável de ambiente

Dúvidas: entre em contato com o administrador do servidor.